Game Foundry Studio

Connected Play / Second Screen Design

Turn-Based TV + Phone/PC Adventure Game Architecture

Recommended file name: GFS-Connected-Play-Second-Screen-Turn-Based-Design-26170-001.pdf

Purpose. This document captures the second-screen game concept discussed for Game Foundry Studio and turns it into a future build note. The goal is to let creators build TV-friendly, turn-based adventure, party, classroom, tabletop, and board-style games where the main game is shown on a TV or shared screen, while players interact from phones, PCs, tablets, or other devices.

Core idea. The TV or host screen is the shared game board. Player devices become private or public controllers. A separate GFS networking server/VPS handles sessions, joining, presence, turn messages, reconnect, and save synchronization.

1. Working Feature Names

- Feature group: Connected Play
- Creator-facing area: Connected Players
- Phone/tablet/PC UI tool: Remote Screens
- Turn flow tool: Turn Manager
- Checkpoint tool: Save Spots
- Backend service name options: GFS Play Server, GFS Session Server, or GFS Networking Server

Recommendation. Use Connected Play as the broad feature name. Use Connected Players as the creator-facing configuration tool. This avoids making the feature sound like normal keyboard/mouse/gamepad controls. The phone can roll dice, spin a wheel, choose a secret card, vote, manage inventory, chat, trade, or operate as a host device.

2. Why This Matters

- Most TVs do not have good keyboard or mouse input, and remote controls are poor for complex game interaction.
- Phones are already available at family game nights, classrooms, parties, and tabletop sessions.
- Turn-based games have very low network traffic: button clicks, dice rolls, card choices, votes, and save events.
- Second-screen input enables hidden information, secret choices, team-only instructions, private inventory, and player-specific prompts.
- This gives GFS a differentiator: creators can build Jackbox-like, Kahoot-like, tabletop assistant, classroom quiz, and social deduction experiences without writing networking code.

3. Example Game Pattern

- A player opens the adventure game on a TV, browser, projector, or large shared monitor.
- The TV shows a QR code and join code.
- Players scan the QR code or visit a join URL from their phone, PC, or tablet.
- Each player joins the session, chooses a name/avatar, and receives a player role.
- The TV displays the shared board, world map, encounter, dice result, wheel animation, or story event.
- Each player device displays the correct UI for that player: roll dice, choose action, vote, pick a card, enter an answer, or manage inventory.
- The networking server validates the action and routes it to the TV/host screen.
- At creator-defined save spots, the session syncs back to GFS storage/database so the group can resume later.

4. Recommended Architecture

Use a separate VPS or server for networking. The main GFS app should remain focused on tools, published games, accounts, project data, and assets. Connected Play should have a separate service boundary because session state, player presence, turn messages, and reconnect behavior are operationally different from static pages and project editing.

Domain / Service	Responsibility	Notes
gamefoundrystudio.com	Main app, tools, creator pages, published game pages	Static app and creator tooling.
api.gamefoundrystudio.com	Accounts, projects, saves, publishing, package metadata	Can be Supabase/API-backed depending on environment.
network.gamefoundrystudio.com	Lobby, sessions, join codes, presence, turn messages, reconnect	Separate VPS/service. This is the Connected Play runtime.
R2 / storage	Assets, project packages, checkpoints, save payloads	Environment prefixes remain DEV/IST/UAT/PROD.

5. Recommended V1 Network Model

Start server-relayed, not peer-to-peer. For turn-based games, the traffic is small enough that phone -> networking VPS -> TV is simpler, more reliable, easier to test, and easier to debug. Peer-to-peer can be added later as an optimization, but it should not block V1.

- V1: Phone/PC/tablet sends action to GFS Play Server.
- GFS Play Server validates session, player, role, and turn ownership.
- GFS Play Server forwards the accepted action to the TV/host screen.
- TV/host screen updates game state and animation.
- At save spots, the game sends a checkpoint to GFS API/storage.

Future peer-preferred mode. After players join through GFS, devices can attempt peer-to-peer connections for gameplay messages. The server would still remain responsible for lobby, signaling,

identity, reconnect, fallback relay, and save sync. If peer-to-peer fails due to NAT/firewall conditions, the game falls back to the server relay.

6. Creator-Facing Tools to Add

Tool	Purpose	Creator Configures	Examples
Connected Players	Defines who can join and what roles exist.	Join code, QR code, player slots, teams, host, spectators, reconnect rules.	Family board game, classroom quiz, party adventure.
Remote Screens	Builds the UI shown on each phone/PC/tablet.	Buttons, dice, wheel, cards, vote, number entry, text entry, inventory, private messages.	Roll dice, choose spell, vote on path.
Turn Manager	Controls turn order and valid actions.	Turn sequence, active player/team, timeout, skip, round rules, host override.	Player 2 rolls, Red Team votes, host reveals answer.
Save Spots	Defines checkpoint and resume behavior.	Save after turn, after round, after room, after boss, manual host save, auto-save interval.	Resume campaign next week from saved room.
Host Controls	Optional game master/teacher/admin controls.	Start round, pause, skip player, award points, reveal card, kick player.	Teacher quiz host, dungeon master, party host.
Presence Monitor	Shows player state to creator/game host.	Connected, disconnected, reconnecting, idle, spectator.	Pause if current player disconnects.

7. Built-In Remote Widgets

- Dice Roller
- Wheel Spinner
- Card Draw / Card Choice
- Vote / Poll
- Yes / No
- Multiple Choice
- Text Entry
- Number Entry
- Slider
- Inventory Selector
- Character Action Buttons
- Team Chat / Private Message

- Timer / Ready Button
- Rating / Stars
- Host Override Buttons

Design principle. Remote widgets should be drag-and-drop building blocks. Creators should not need to know WebSockets, session IDs, JSON payloads, or networking details. The engine should turn widget actions into game events.

8. Core Game Events

- On Session Created
- On Player Join
- On Player Leave
- On Player Reconnect
- On Player Ready
- On Turn Start
- On Turn Timeout
- On Player Action
- On Vote Complete
- On Dice Rolled
- On Wheel Spun
- On Card Chosen
- On Host Action
- On Save Spot Reached
- On Save Complete
- On Network Error

9. Roles and Permissions

Role	Can Do	Use Cases
Host	Create/start session, pause, skip, reveal, award, save, end session.	Teacher, party host, dungeon master, parent.
Player	Interact when allowed, receive private screens, take turns, vote, choose actions.	Normal adventure/party game participant.
Team Member	Act for team or vote with team.	Team-based puzzles, classroom groups.
Spectator	View only, no gameplay actions unless promoted.	Audience, stream viewers, classroom observers.
Creator/Test User	Debug session, inspect messages, simulate players.	Creator testing during build.

10. Private Information Support

Private player UI is a major reason to build this feature. The TV can show shared information while each player device shows information only that player, team, or host should see.

- Secret missions: only one player sees their objective.
- Hidden roles: social deduction games where players have private roles.
- Card hands: each player sees their own cards.
- Inventory: each player sees personal items and abilities.
- Team-only instructions: Red Team and Blue Team receive different prompts.
- Puzzle input: one player reads clues while another sees controls.

11. Turn-Based Validation Rules

- Only the current player or current team can submit turn actions unless the creator allows all-player input.
- The server should reject duplicate, late, invalid, or out-of-turn actions.
- Actions should include a session ID, player ID, action type, action payload, and turn/round counter.
- Server should provide idempotency keys so a double tap does not roll dice twice.
- The TV/host should receive accepted actions, not raw untrusted inputs.
- Host override can skip, pause, undo, or force a state depending on creator settings.

12. Save Spots / Checkpoints

Save Spots are creator-defined moments where the game reconnects/syncs to GFS for durable storage. This keeps live turn traffic on the networking server while project saves and resumable campaign state remain under the normal GFS data/storage model.

- Save after each turn.
- Save after each round.
- Save after each room/level/map.
- Save after boss or story milestone.
- Save when host presses Save.
- Auto-save every X minutes.
- Save only when all players confirm.
- Save on graceful session end.

Recommended checkpoint contents. Current map/room, player positions, player roles, player inventory, score, quest flags, turn order, round number, random seed, unresolved prompts, disconnected player state, and last accepted action ID.

13. Session Lifecycle

1. Creator configures Connected Play for the game.
2. TV/host launches the published game.
3. TV requests a new session from network.gamefoundrystudio.com.
4. Network server returns session ID, short join code, and QR join URL.

5. Players join from phones/PCs/tablets.
6. Host starts the game when enough players are ready.
7. Turn Manager selects current player/team.
8. Remote Screens display only the allowed controls for that player/team/host.
9. Player submits action.
10. Network server validates and routes action.
11. TV/host applies action to game state and shows animation/result.
12. Save Spot triggers checkpoint sync to GFS API/storage.
13. If a player disconnects, Presence Service marks them disconnected and supports reconnect.
14. At session end, final checkpoint is saved and session is closed.

14. Backend Services on the Separate VPS

Service	Responsibility	Notes
Lobby Service	Create sessions, join codes, QR URLs, session capacity, host ownership.	First service needed for V1.
Presence Service	Track connected, disconnected, reconnecting, idle, ready.	Needed for turn reliability.
Turn Service	Validate active player/team and accepted action sequence.	Critical for turn-based games.
Message Service	Route phone/PC actions to TV/host and host messages to players.	Likely WebSocket-based for V1.
Remote Screen Service	Serves or describes the correct UI state for each device.	Can be driven by the game definition.
Save Sync Service	Sends checkpoint payloads to GFS API/storage.	Bridge between networking VPS and GFS project/save model.
Reconnect Service	Allows a player to return to their slot after network loss.	Use session token or rejoin code.
Telemetry/Health	Tracks service status, session counts, errors, latency.	Useful for owner operations.

15. Suggested Message Types

- session.create
- session.join
- session.leave
- session.ready
- player.assign
- turn.start
- turn.timeout

- action.submit
- action.accepted
- action.rejected
- dice.roll
- wheel.spin
- vote.cast
- vote.complete
- card.choose
- host.pause
- host.skip
- host.award
- save.request
- save.complete
- presence.update
- error.network
- error.invalid_turn

16. GFS Game Journey Placement

Do not hide this under normal Controls only. Keyboard, mouse, gamepad, touch, and accessibility are device inputs. Connected Play is a game mode and session architecture.

- Recommended placement: Interface -> Connected Players -> Rules
- Alternative placement: Controls -> Connected Players -> Rules, if the toolbox needs it near input configuration.
- The feature should still connect to Controls for accessibility and input mapping, but it should have its own tool group or tile.

17. MVP Scope

- Create session from TV/host screen.
- Show QR code and short join code.
- Allow players to join from phone/PC/tablet.
- Assign host and player roles.
- Show basic remote screens: button, dice, wheel, vote, multiple choice.
- Support turn order with current-player validation.
- Route action from player device to TV/host screen through networking VPS.
- Show presence: connected/disconnected/rejoined.
- Support manual save and save-after-turn.
- Support reconnect to same player slot.

18. Later Enhancements

- Peer-preferred mode using WebRTC for browser-to-browser data channels, with server fallback.
- Local Party mode for same-Wi-Fi sessions.

- Team-only screens and team chat.
- Spectator view and stream-friendly audience mode.
- Advanced hidden roles and secret objectives.
- Companion UI templates for board games, quizzes, tabletop adventures, and social deduction games.
- Creator test harness to simulate 2, 4, 8, or more players from one browser.
- Replay log and action history for debugging.
- Session analytics: average players, disconnects, turn duration, abandoned sessions.
- Moderation tools for public sessions.

19. Risks and Design Notes

- Security: the server must not trust client actions. Validate player, role, turn, action type, and payload.
- Double actions: use action IDs/idempotency keys to prevent double rolls or duplicate votes.
- Reconnect: disconnected players need a safe way to reclaim the same slot.
- Privacy: private screens must not leak hidden cards, roles, objectives, or team prompts to the TV or wrong player.
- Scalability: turn-based traffic is small, but public games and classroom sessions may create many idle WebSocket connections.
- Cost: V1 relay is simpler but uses server bandwidth. For turn-based games this should be manageable.
- Fallback: peer-to-peer should be optional later, not required for reliability.
- Debugging: creators need clear test panels, not invisible networking failures.
- Save authority: GFS should remain the durable save/checkpoint authority even when live gameplay traffic uses the networking server.

20. Example Creator Configuration

Setting	Value
Game Type	Turn-based adventure board game
Players	2-6 players, 1 host optional, spectators allowed
Join Method	QR code + 6-digit join code
Remote Widgets	Dice, choose path, use item, vote, ready button
Turn Rule	Clockwise player order; skip after timeout; host can override
Private UI	Each player sees inventory and secret objective
Save Spots	After each room and manual host save
Reconnect	Player can rejoin same slot with session token or join code + name confirmation
Networking Mode	V1 server relay; future peer-preferred optional

21. One-Line Build Summary

Connected Play lets creators build TV-first, turn-based games where the shared screen is the adventure board and each player device becomes a private, role-aware controller, all powered by a separate GFS Play Server for sessions, presence, turn validation, messages, reconnect, and save synchronization.